Code Jam 2013 - World Finals

Analysis: Can't Stop

Divide and Conquer

There is a nice divide-and-conquer solution. Split the input down the middle; the best interval is either entirely in the left half, entirely in the right half, or crosses the middle. We handle the right and left cases recursively; if we can handle the "crosses" case in linear time, we will have an `O(N lg N)` algorithm, which is good enough.

If the best interval crosses the middle, we know it must use the middle element, so we must pick one of the **D** numbers in that roll to use. Now go out as far as we can to the left and right with that number. Once we stop, we can extend either to the left or the right, so there are `2D` different numbers to try. Expand again, and get `2D` more numbers to try for a final expansion. In total, we only tried `D*2D*2D` different choices, so the whole thing runs in `256N` time, and we get our `O(N lg N)` time algorithm.

Note: surprisingly for a divide and conquer algorithm, we don't use any information from the left and right halves to solve the middle case; the fact that it crosses is enough.

Slow and Simple

There's another set of approaches to this problem. Try every starting position, expanding out to the right and only making choices when necessary (just as in the divide-and-conquer). This is $O(D^K * N)$ for each starting position, or $O(D^K * N^2)$ overall, which is too slow. However, there are several different improvements (some small, some large) one could make in order to make this linear in N.

Linear Time

One improvement works in this way: Every time you pick a number, check if the roll before your starting position contains that number. If so, we can safely ignore that choice (because we would have done better starting one roll earlier and choosing the same set of numbers).

It turns out that this runs in linear time, but the reason why isn't obvious. We want to prove that a particular index `i` is only visited from `O(1)` starting positions. Imagine we have reached `i` from `start`. Now imagine starting at `i` and going leftward (as usual, we expand left as far as possible before picking each new number). If we only choose numbers from the set that got us from `start` to `i`, we must get back to `start` (and no further, since the optimization above guaranteed that `start - 1` doesn't contain any of our numbers). But there are only $1 + D + D^2 + D^3$ different choices of numbers starting from `i`, so there are only that many possible positions for `start`, so `i` will only be visited $O(D^3)$ times.

Code Jam 2013 - World Finals

Analysis: Drummer

This problem can be viewed as a modified version of finding the minimum width of a convex hull [1] which can be solved using a modified version of the rotating calipers technique [2,3]. We present below the intuition on why the problem is similar to finding the modified minimum width of the convex hull.

First off, we can plot the sequence of drum strikes as (i, T_i)on the 2-dimensional plane. If the drummer performs perfectly, then let's call the sequence P_i and (i, P_i) falls on the same line L (see *Figure 1* and *Figure 2*).

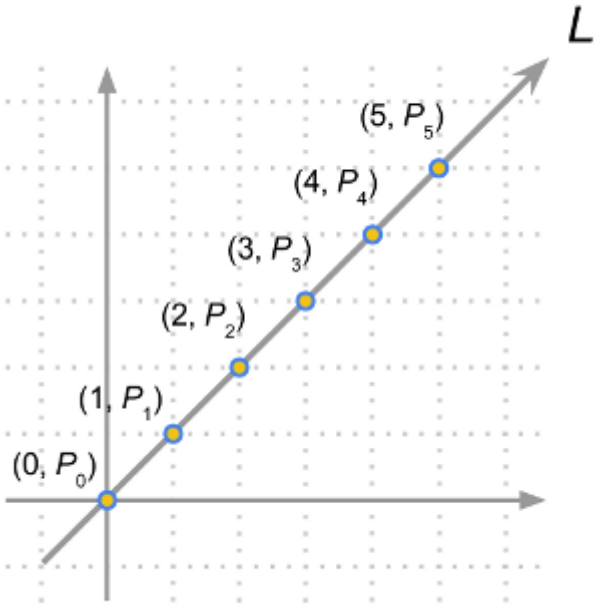


Figure 1

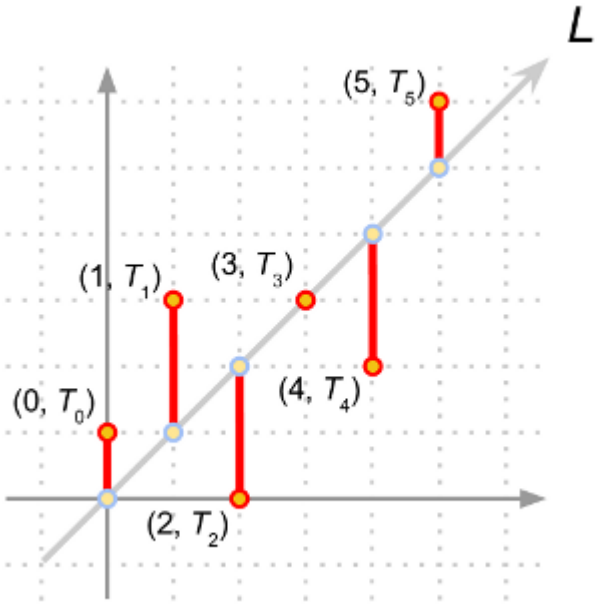


Figure 2

Unfortunately the drummer does not perform perfectly and has error E_i (which is $|T_i - P_i|$) (see Figure 3).

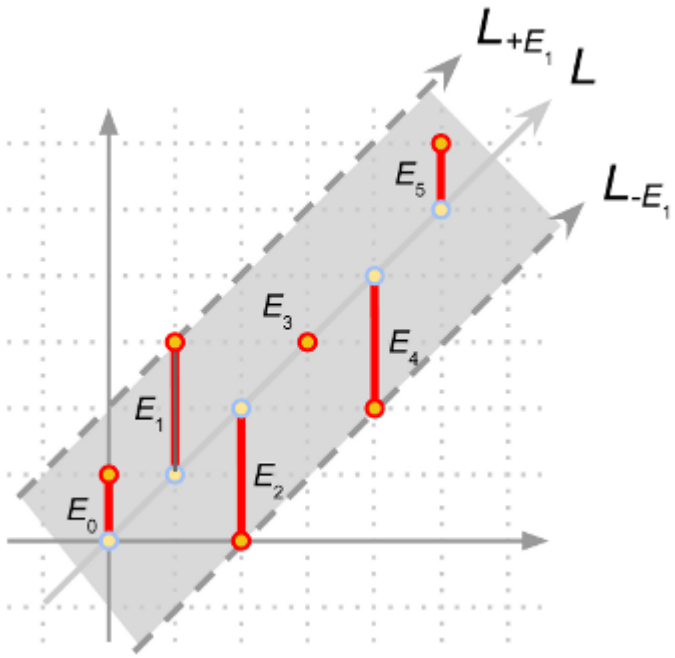


Figure 3

So in the example in Figure 3, we have our line L, and it is clear that our answer is the maximum among all E_i, which in this case is E_1 (or E_2 or E_4). If we draw two lines parallel to L shifted in the y-axis by $+E_1$ and $-E_1$, denoted as L_{+E_1} and L_{-E_1} (see *Figure 3*), you will notice that the region between the two lines contains all the points (i, T_i); this example fits the definition of the problem which is a drum rhythm where each strike differs by at most E from some perfect rhythm. On the other hand, if we choose E_5 as the error candidate, the region between the two lines L_{+E_5} and L_{-E_5} will not contain all the points (i, T_i) (see *Figure 4*).

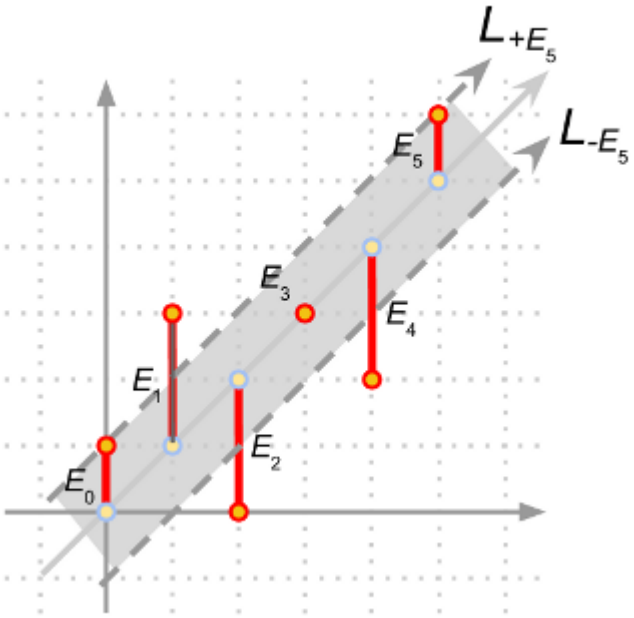


Figure 4

In essence, we want to find a line denoting the perfect rhythm and two parallel lines that are shifted in the y-axis by *+E* and *-E* such that the two parallel lines contain all the points (i, T_i) and also that *E* is the minimum possible value. We would like to point out that instead of trying to find the line with the perfect rhythm which can be a hard problem due to the error associated with each (i, T_i), we can instead focus on the equivalent problem of finding the two parallel lines. Note that each of these two lines must touch at least one point otherwise we can always reduce the distance between these two lines, and further reduce the distance on y-axis to get a smaller error *E*.

In fact, all candidate parallel lines touch (without intersecting) the convex hull of the points (i, T_i). Therefore we transform the problem to finding the minimum distance between two parallel lines in y-axis touching (without intersecting) the convex hull. For example, the convex hull of the points in *Figure 2* are shown in *Figure 5*.

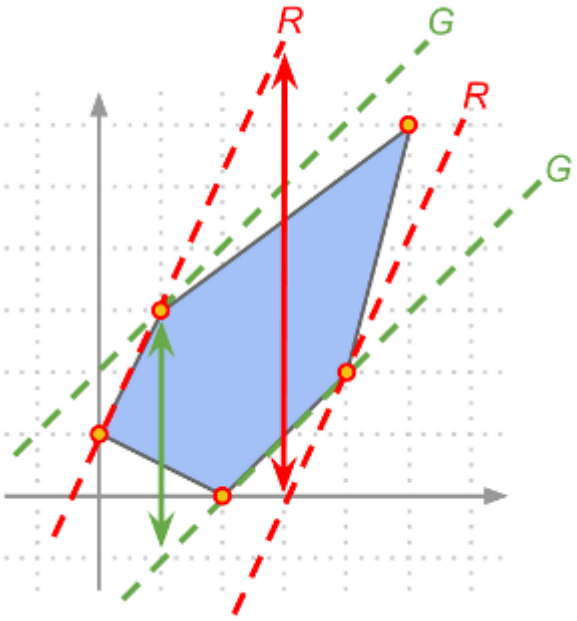


Figure 5

Therefore to solve this problem, we first compute the convex hull of the points (i, T_i). Then we go through all the line segments on the boundary of the convex hull and find the corresponding parallel line on the opposite side of the convex hull (see *Figure 5* for some examples), then get the y-distance between these two parallel lines, and report the minimum among all such y-distances.

Code Jam 2013 - World Finals

Analysis: Graduation Requirements

In this problem we are interested in finding the maximum time we can drive against the direction in a traffic circle. For small test cases, the size of the input is small enough to try all possible intersections, time to start, and check the length to drive without touching any other car.

For large test cases, the above approach is not good enough due to the extremely large **N** and **X** (up to 10^{10}). Therefore in order to solve the large test cases, we transform the problem into the 2-dimensional plane with intersection and time as axes. Then, each car can be represented as line segments; your car will be represented as a line segment orthogonal to other line segments formed by other cars. Figure 1 shows the second sample test case. Lined segments corresponding to different cars are denoted with different colors while the black dotted segment corresponds to your car. Note that as the intersections are in a traffic circle, we show it by repeating intersection 5 and 1 on the two ends in Figure 1.

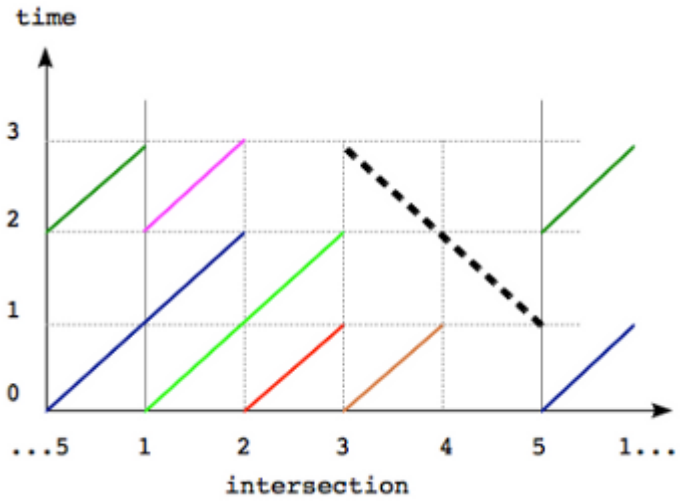


Figure 1

So the problem can be reformulated as follows: given a set of line segments, find the maximum possible length of a perpendicular segment which does not touch any other segment. Note that if two segments have a point in common the corresponding cars touch each other.

To solve the large test case one needs to notice that a line segment of maximum possible length can be chosen so that it goes near one of the endpoints of another segment. By â€œnearâ€ we mean distance = 1 in one of the axes. Indeed, if we have a segment of maximum possible length, that does not go near one of the endpoints, we can always move it so that it does, see Figure 2 for an example.

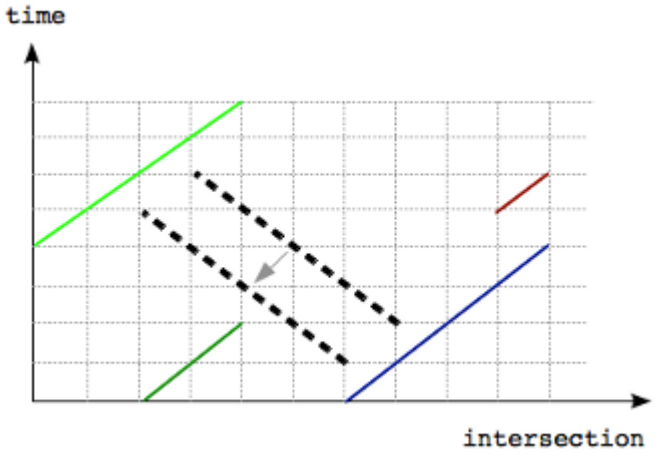


Figure 2

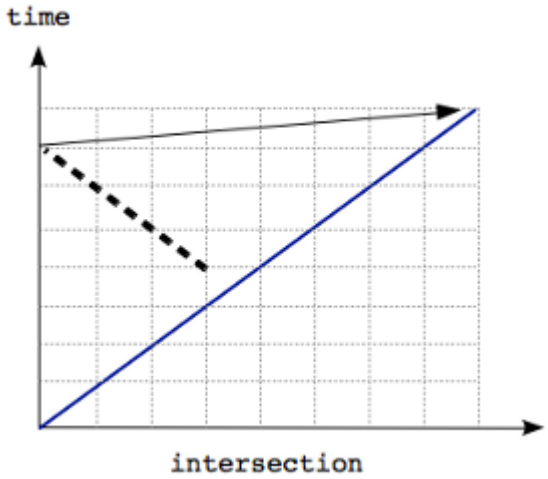


Figure 3

The statement is also true in examples such as in Figure 3 because intersections are arranged in a circular manner and the solution segment is near the top right endpoint which is shown with an arrow.

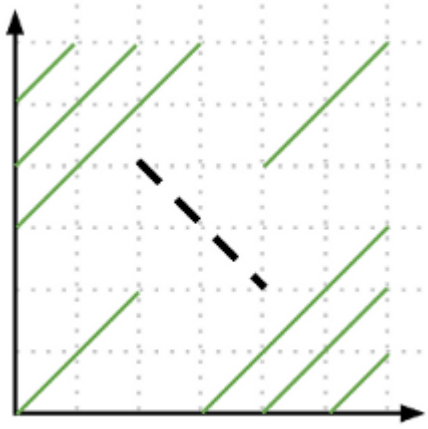


Figure 4

Thus we note that changes in the length of segments happen near these endpoints. Therefore to solve the problem we need to go over all endpoints of car segments and consider its neighbourhood (-1 and +1 in both of the axes). Note that we also need to consider segments that go through endpoints +/-(1,1), an example is shown in Figure 4. Then we can compute all segments that pass through these near points and pick the longest segment as the answer.

For each such candidate point, we need to check how far up and down a segment that go through the candidate point can reach without touching other segments. This can be done simply by going over all car segments and checking if and where our candidate segment intersect them. The complexity of this algorithm is $O(C^2)$, which is enough to solve the large case.

Code Jam 2013 - World Finals

Analysis: Let Me Tell You a Story

Dissecting the problem

The problem statement asks for the number of ways to remove elements from a sequence until we obtain a non-increasing sequence. Looking at the problem from the end, suppose we know the final non-increasing sequence, and it's of length **K**. Certain elements have been eliminated, and we removed **N**-**K** elements in total. There are (**N**-**K**)! (factorial of **N**-**K**) ways to remove those elements. So the first approximation seems to be to sum those factorials over all non-increasing subsequences of the the original sequence.

However, this is not entirely correct. Some non-increasing subsequences are not even reachable since we stop as soon as our sequence becomes non-increasing. For example, in the second example from the problem statement the sequence is non-increasing from the start, so no proper subsequence is reachable at all. For subsequences that are reachable, not every way of reaching them might be possible. For example, in the first example from the problem statement we can reach the '7 <first 6>' subsequence (note, it should be considered different from '7 <second 6>' subsequence) by eliminating the second 6, and then 4. However, it can't be reached by doing those operations in reverse order, since we'd stop right after eliminating 4.

Now instead of all (**N**-**K**)! ways to reach a certain subsequence, we need to count just the possible ways. The main trick in solving this problem is: let's count the impossible ways instead, and then subtract. The impossible ways are simply those when before the last removal, the subsequence was already a non-increasing sequence of length **K**+1. And for each such subsequence there are exactly **K**+1 ways to make an impossible removal and arrive at a subsequence of length **K**. That means the sum of numbers of impossible ways to reach all non-increasing subsequences of length **K** is equal to the total number of non-increasing subsequences of length **K**+1 times (**N**-**K**-1)! (the number of ways to reach the longer subsequence) times **K**+1.

To summarize, suppose $\mathbf{A_K}$ is the number of non-increasing subsequences of length **K**. Then the answer to this problem is sum over **K** of $\mathbf{A_K}$*(**N**-**K**)!-$\mathbf{A_{K+1}}$*(**N**-**K**-1)!*(**K**+1).

Solving the sub-problem

We've now reduced our problem to a much simpler one: find $\mathbf{A_K}$. This problem can be solved used a somewhat standard dynamic programming approach, with a twist to make it run faster.

First, let's assume that all input numbers are different, and between 0 and **N**-1. It's not hard to transform them in this way without changing the answer. If there are equal numbers, we'll slightly reduce the number to the right, so that non-increasingness of all subsequences is preserved.

Our dynamic programming problem will now be: what is the number of non-increasing subsequences of length **P** that end with number **Q**? Let's call that number $\mathbf{B_{P,Q}}$. We will find them in the order **Q**s appear in the sequence.

It's not hard to see that $\mathbf{B_{P,Q}}$ is just the sum of $\mathbf{B_{P\text{-}1,Q'}}$ for all numbers **Q'** that are greater than **Q** and appear before **Q** in the sequence. Since we process the states in the order **Q**s appear in the sequence, we just need to take the sum over all **Q'** that are greater than **Q**.

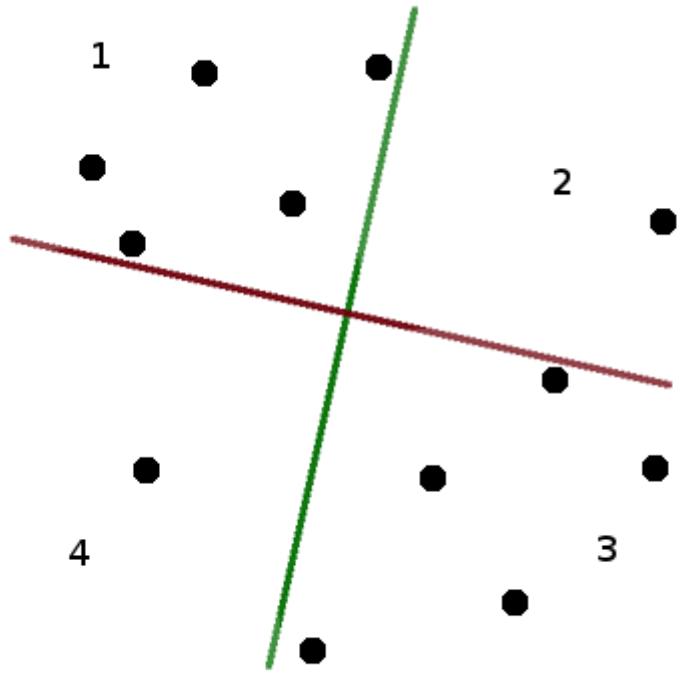
This is already a working solution for our problem, but it is a bit too slow: it runs in $O(\mathbf{N}^3)$ which is a bit too much for **N**=8000. We have $O(\mathbf{N}^2)$ states in our dynamic programming, and we need to find a sum of $O(\mathbf{N})$ numbers to process each state. However, we can compute such sum faster! We just need an array-like data structure that supports changing elements and finding the sum of its suffix (over all **Q'** that are greater than **Q**), and the Fenwick tree is a data structure that does exactly that, performing each operation in $O(\log\mathbf{N})$ time, for a total running time of $O(\mathbf{N}^2*\log\mathbf{N})$.

Code Jam 2013 - World Finals

Analysis: X Marks the Spot

Median choice

Since the four quarters formed by the two lines have to contain the same number of mines, each of the two lines has to split the set in half. Thus, if we know the inclination of the two lines (given as, e.g., the directed angle the first line forms with the horizontal axis), we can position the each line in any place such that it splits the points in half (by, for instance, sorting the points by the value of the cross-product with the direction of the line). Let's begin by choosing any angle alpha as our initial angle, and draw the two lines according to the procedure above.

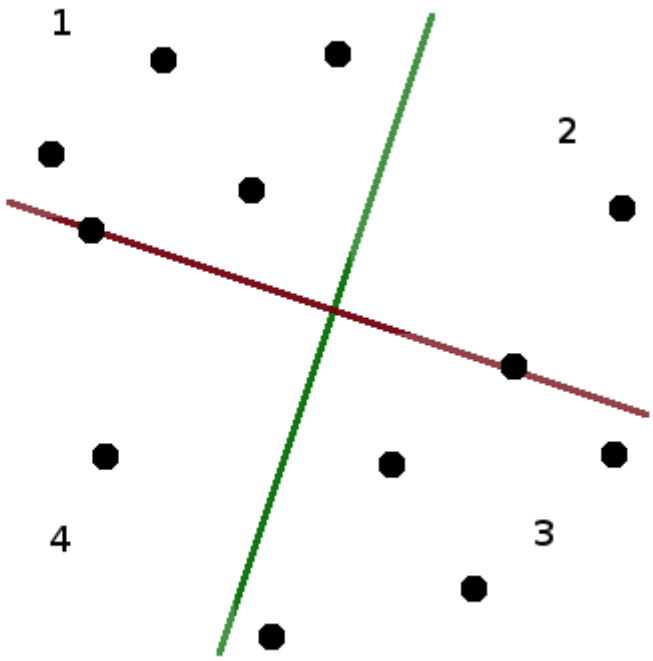


Suppose that the first quarter contains **X** points. The second quarter contains 2**N** - **X**, since there have to be 2**N** points above the red line. The third quarter will again contain **X** points (because there are 2**N** points to the right of the green line), and the fourth will contain 2**N** - **X**. So if **X** = **N**, our two lines are a correct solution. This doesn't need to be the case, though, as we can see on the figure above.

Rotating the lines

If the angle alpha we chose happens not to be a correct solution to the problem, we will try rotating the lines (by increasing alpha) until it becomes valid.

Let's consider what happens when we rotate the lines. At some moment, one or both of the lines will rotate to a point where instead of splitting the set neatly in half, the "splitting line" passes through two mines, with 2**N**-1 mines on either side (note that we are taking advantage of the fact that no three points are collinear, so we know the line passes through exactly two points, and not, say, four). We will call the moment at which there are two points on at least one of the splitting lines a "discontinuity". After we rotate a tiny bit more, the lines split the set neatly again.

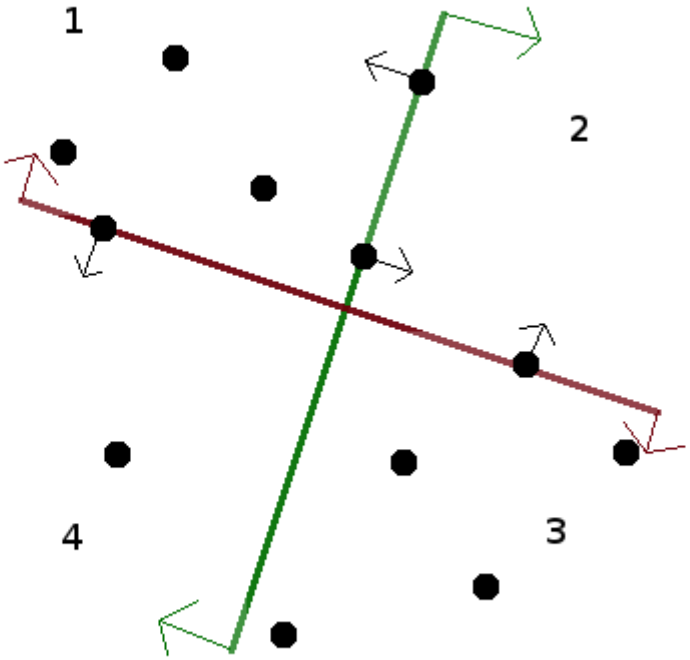


We will prove in a moment that at any discontinuity, **X** changes at most by one (it can also happen to stay the same). Notice, however, that when we increase alpha by 90 degrees, the red and green lines will exchange places, which means that the first quarter (which also rotated by 90 degrees) now contains 2**N**-**X** points. Thus, somewhere in between **X** had to be exactly equal to **N**!

Discontinuity

Let's analyze what happened after the discontinuity. Obviously, only the points that were on the dividing lines could have changed quarters at the discontinuity. One of the points that were on the red line crosses from one of quarters (1, 2) to one of quarters (4, 3), and the other point crosses in the other direction. Thus, if the discontinuity had points on one line only, **X** changes by at most one.

We will see that even if there were points on both lines at the discontinuity, **X** will still change only by one. The points on the red line go from quarters (1, 4) to quarters (2, 3). So, for **X** to, say, grow by two, we would have to have a point from 2 go to 3 and point from 4 go to 1 on the red line, while a point from 4 went to 3 and a point from 2 went to 1 on the green line. However, the red line (and the green line as well, but for now it's the red line that matters) is rotating clockwise - thus, it's the more leftward point that will go down, and the more rightward point that will go up — so the situation described above is impossible.



Finding the solution

We can now use binary search to find a solution to the problem. Begin with an arbitrary angle alpha as the left bound, and alpha + 90 degrees as the right bound, assume that for the left bound **X** is smaller than **N** (if it's equal, we're done, and if it's larger, take alpha + 90 as left and alpha + 180 as right). We know somewhere between the two there is a point in which **X** = **N**. So we pick an angle midway between left and right, and check how big is **X** for this median angle. If it's equal to **N**, we are done. If it's smaller, the median angle is our new left, if it's larger, it's the new right. Proceed until success.

A single iteration, with the standard implementation, takes O(**N** log**N**) time - sort the points twice, assign each to the appropriate quarter, find **X**. If somebody cared enough (one doesn't need to in this problem) it can be done in O(**N**) time, using a faster algorithm to find the median values (either a randomized one, like quicksort, but recursing only into the bigger of two halves, or even deterministically with the median of medians algorithm.

The key question is "how many iterations of the binary search algorithm will we need to find the angle we are looking for?". This depends on the size of the interval of angles that we will try to hit. This interval will occupy the space between some two discontinuities, and each discontinuity is defined by a line connecting two of the input points - thus, the size of the interval can be expressed as an angle between two lines, each crossing two points with integral coordinates no larger than 10^{-6}. Such an angle can be on the order of 10^{-12}, which means we will need roughly 40 steps of the binary search. Thus, our algorithm will easily run in time.

Precision issues

There are three types of precision issues that can hit one on the solving of this problem.

First, it can happen that one of the lines we choose happens to be a discontinuity. This can be either worked around (by choosing a median angle a bit to the left or right - there are only finitely many discontinuities, after all), or avoided by choosing a random angle to begin with - since

there are finitely many discontinuities, it's very unlikely to hit one. It's also possible to choose a deterministic angles to avoid discontinuities.

Second, the interval of angles that we are trying to find can happen to be rather small, and so we will need many iterations of the binary search. This means that we need to use relatively high precision numbers to deal with the quantities involved. It's a pretty standard issue in geometry problems, though, and shouldn't surprise anyone.

Third, there are limits on the precision with which we can output the result, and the checker for the problem will check whether the output values are correct. Since there is a finite precision of the output, there will be some rounding happening. This means that if we were unlucky, and our chosen angle happened to be quite close to the boundary of the "good" interval, the rounding can push it out of the interval. There are two things we can do to mitigate this. First, we can add a few more steps of the binary search to find a few more points within the good interval, and then choose for our answer a point that we know is relatively far from the edge. Second, we should use all the precision that we are allowed in the input. In particular, when we know the line we want to draw, we should choose the second point we output (the one other than the crossing) to be as far away from the crossing as the limits on the output allow us, to minimize the error in the angle of the line resulting from rounding.